CSA5112 - Cryptography and Network Security

14. Elliptic Curve Cryptography (ECC)

Aim: To implement the Elliptic Curve Cryptography (ECC) algorithm for encrypting and decrypting a message using public-key cryptography.

Algorithm:

1. Start.
2. Import the required ECC libraries.
3. Generate an ECC public and private key pair.
4. Read the plaintext from the user.
5. Encrypt the plaintext using the public key.
6. Decrypt the ciphertext using the private key.
7. Display the decrypted message.
8. Stop.

Code:

```
from Crypto.PublicKey import ECC

from Crypto.Signature import DSS

from Crypto.Hash import SHA256

# Generate ECC Key Pair

key = ECC.generate(curve='P-256')

private_key = key

public_key = key.public_key()

print("ECC Key Pair Generated Successfully.\n")

message = input("Enter Message: ")

hash_obj = SHA256.new(message.encode())

# Sign

signer = DSS.new(private_key, 'fips-186-3')

signature = signer.sign(hash_obj)
```

```
print("\nDigital Signature (Hex):")
print(signature.hex())
# Verify
verifier = DSS.new(public_key, 'fips-186-3')
try:
    verifier.verify(hash_obj, signature)
    print("\nSignature Verified Successfully.")
except ValueError:
    print("\nSignature Verification Failed.")
```

Sample Input:

```
Enter Message:
HELLO WORLD
```

Sample Output:

```
ECC Key Pair Generated Successfully.

Enter Message:
HELLO WORLD

Digital Signature (Hex):
7b7d5f62a1d9b0c8ef2f8b8f8e58d31b6b5ad0d3...

Signature Verified Successfully.
```

Result:

The Elliptic Curve Cryptography (ECC) algorithm was successfully implemented by generating an ECC key pair, creating a digital signature for the given message, and verifying the signature successfully.